

ESP-IDF Build & Flash

ESP-IDF Build & Flash

- 1. Overview
- 2. Prerequisites
- 3. VSCode GUI Build & Flash
 - 3.1 Bottom Toolbar
 - 3.2 Operation Steps
 - Step 1: Select Target Chip
 - Step 2: Select Serial Port
 - Step 3: Build
 - Step 4: Flash
 - Step 5: Serial Monitor
 - Keyboard Shortcuts Summary
- 4. Command Line Build & Flash
 - 4.1 Launch ESP-IDF Terminal
 - 4.2 Common Commands
 - Set Target Chip
 - Build
 - Flash
 - Serial Monitor
 - One-Click Operation
 - Clean Build Outputs
 - View Project Configuration
- 5. Flash Download Tool
 - 5.1 Installation
 - 5.2 Operation Steps
 - Step 1: Launch Tool
 - Step 2: Configure Flash Files
 - Step 3: Configure Flash Parameters
 - Step 4: Connect & Flash
 - Step 5: Manually Enter Download Mode
 - 5.3 Merged Firmware Flashing
- 6. Flash File Description
- 7. Troubleshooting

1. Overview

An ESP-IDF project goes through two steps from source code to running on a development board: **Build** and **Flash**. This article covers three operation methods:

Method	Use Case
VSCode GUI	Daily development, one-click build/flash/monitor
Command Line <code>idf.py</code>	CI/CD, scripting, remote terminal

Method	Use Case
Flash Download Tool	Flashing only pre-compiled .bin firmware, no source environment required

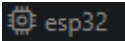
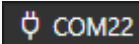
2. Prerequisites

- ESP-IDF environment is installed (refer to **Environment Setup (Windows)**)
- USB data cable connected to ESP32-S3 development board
- Device Manager recognizes the serial port (e.g., COM3)
- Drivers installed (CP210x / CH340 / CH343 / FT232)

3. VSCode GUI Build & Flash

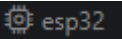
3.1 Bottom Toolbar

After installing the ESP-IDF extension in VSCode, the bottom status bar will show ESP-IDF toolbar icons:

Icon	Function	Description
	Target Chip	Displays current chip model (e.g., esp32s3)
	Serial Port	Select the development board's COM port
	Build	Build the current project
	Flash	Flash compiled output to the development board
	Monitor	Open serial monitor to view log output
	One-Click Operation	Execute sequentially: Build → Flash → Monitor

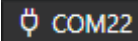
3.2 Operation Steps

Step 1: Select Target Chip

Click the chip  selection area on the bottom status bar → Select `esp32s3`.

ESP-IDF will regenerate `sdkconfig` to adapt to ESP32-S3.

Step 2: Select Serial Port

Click the serial port area  on the bottom status bar → Select the development board's corresponding COM port.

The bottom status bar should display:

[ESP-IDF]	esp32s3		COM3		...
-----------	---------	--	------	--	-----

Step 3: Build

Click the  **Build** icon (or press `Ctrl + E, B`).

- Initial build: approximately 2~5 minutes (all components need to be compiled)
- Incremental build: only a few seconds (only modified files are recompiled)

The end of the output should display:

```
Project build complete.
```

Build outputs are located in the `build/` folder under the project directory:

File	Description
<code>bootloader/bootloader.bin</code>	Secondary bootloader
<code>partition_table/partition-table.bin</code>	Partition table
<code>*.bin</code>	Application firmware
<code>flash_args</code>	Flash parameter file

Step 4: Flash

Click the  **Flash** icon (or press `Ctrl + E, F`).

The flashing process automatically performs the following:

1. Put the chip into download mode
2. Write `bootloader.bin` → `partition-table.bin` → `app.bin` sequentially according to the partition table
3. After flashing completes, the chip automatically resets and the program starts running

Step 5: Serial Monitor

Click the  **Monitor** icon (or press `Ctrl + E, M`).

The OUTPUT panel will display the chip's serial logs. Press `Ctrl + J` to exit monitoring.

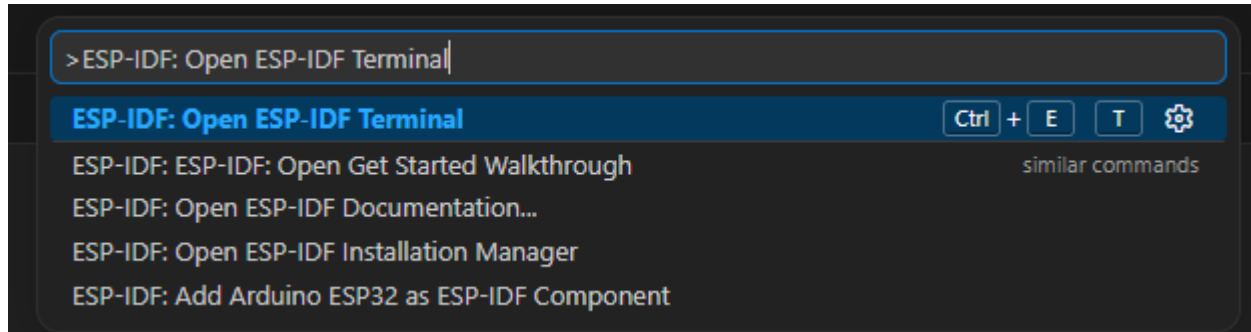
Keyboard Shortcuts Summary

Shortcut	Function
<code>Ctrl + E, B</code>	Build
<code>Ctrl + E, F</code>	Flash
<code>Ctrl + E, M</code>	Monitor
<code>Ctrl + E, X</code>	One-click: Build + Flash + Monitor

4. Command Line Build & Flash

4.1 Launch ESP-IDF Terminal

ESP-IDF commands must be run in the **EIM Terminal** (with ESP-IDF environment variables loaded). **F1** → **ESP-IDF: Open ESP-IDF Terminal**



4.2 Common Commands

Set Target Chip

```
idf.py set-target esp32s3
```

Only execute when first setting up or switching chip models. This will regenerate `sdkconfig`.

Build

```
idf.py build
```

Flash

```
idf.py flash
```

Specify serial port and baud rate:

```
idf.py -p COM3 -b 115200 flash
```

Parameter	Description
<code>-p COMx</code>	Specify serial port number
<code>-b 115200</code>	Specify flash baud rate

Serial Monitor

```
idf.py monitor
```

Exit monitor: `Ctrl + ]`

One-Click Operation

```
idf.py build flash monitor
```

Clean Build Outputs

```
idf.py fullclean      # Full clean (everything except sdkconfig)
idf.py clean          # Clean build directory only
```

View Project Configuration

```
idf.py menuconfig
```

5. Flash Download Tool

The ESP Flash Download Tool (Windows GUI tool) is suitable for **flashing pre-compiled .bin firmware without installing the ESP-IDF environment**.

5.1 Installation

Simply extract 6. `软件工具->固件烧录工具->flash_download_tool.zip`, no installation required. After extraction, double-click `flash_download_tool_x.x.x.exe` to use it.

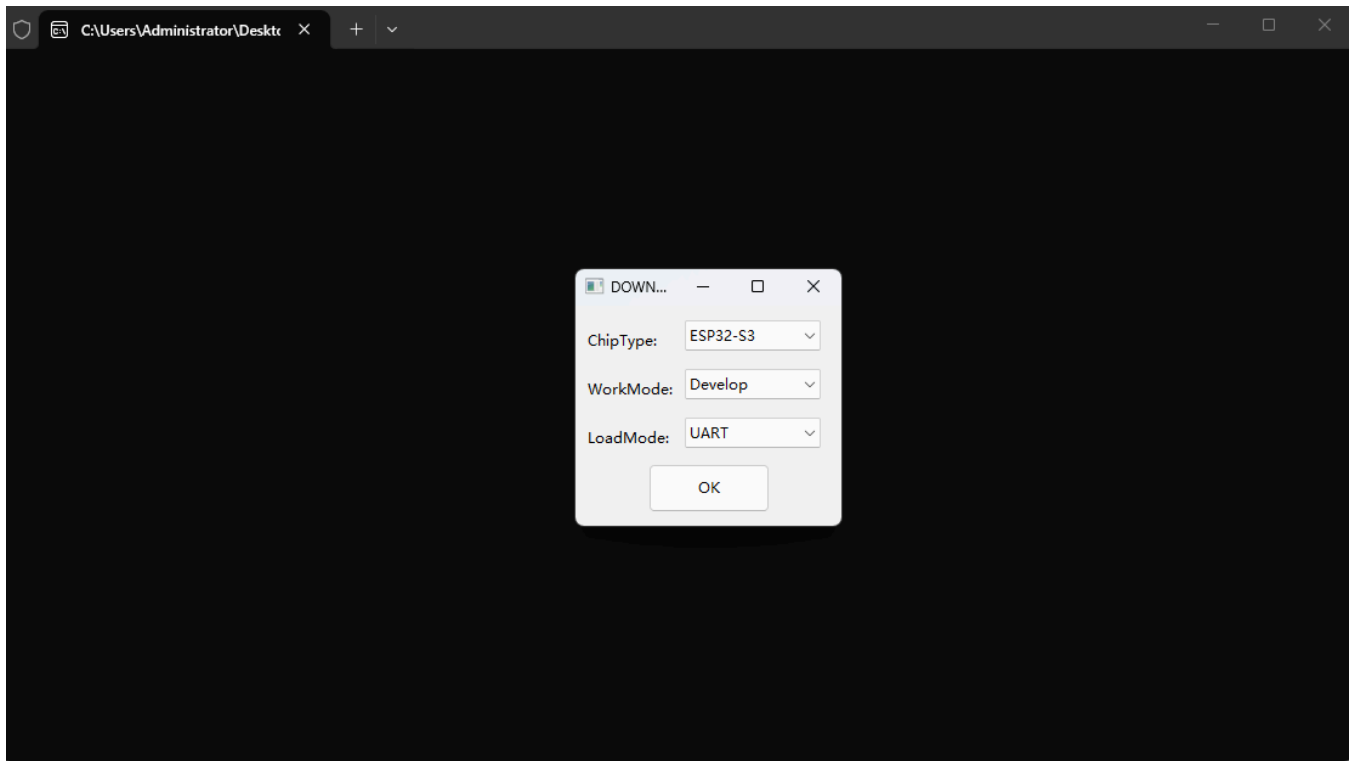
5.2 Operation Steps

Step 1: Launch Tool

Double-click `flash_download_tool_x.x.x.exe` → Select chip model `ESP32-S3` → Confirm.

Step 2: Configure Flash Files

In the flashing interface, add firmware files and fill in the corresponding flash addresses:



File	Address	Description
bootloader.bin	0x0	Secondary bootloader
partition-table.bin	0x8000	Partition table
ota_data_initial.bin	0xd000	OTA state partition initialization firmware
AUDIO_PLAYER.bin	0x10000	Main application firmware

File paths cannot contain Chinese characters or spaces, otherwise the tool will fail to parse.

```
正在执行任务: C:\Espressif\tools\python\v6.0.2\venv\Scripts\python.exe D:\espidf\6.0.2\espressif\v6.0.2\esp-idf\components\esptool_py\esptool\esptool.py -p COM22 -b 460800 --before default-reset --after hard-reset --chip esp32s3 write flash --flash_mode dio --flash_freq 80m --flash_size 16MB 0x0 bootloader/bootloader.bin 0x8000 partition table/partition-table.bin 0xd000 ota_data_initial.bin 0x10000 AUDIO_PLAYER.bin
```

Check the checkbox before each file to activate that flash item.

Step 3: Configure Flash Parameters

SPI SPEED

☐ 40MHz
 ☐ 26.7MHz
 ☐ 20MHz
 ☒ 80MHz

SPI MODE

☒ QIO
 ☐ QOUT
 ☐ DIO
 ☐ DOUT
 ☐ FASTRD

☐ DoNotChgBin
 ☐ LockSettings

CombineBin

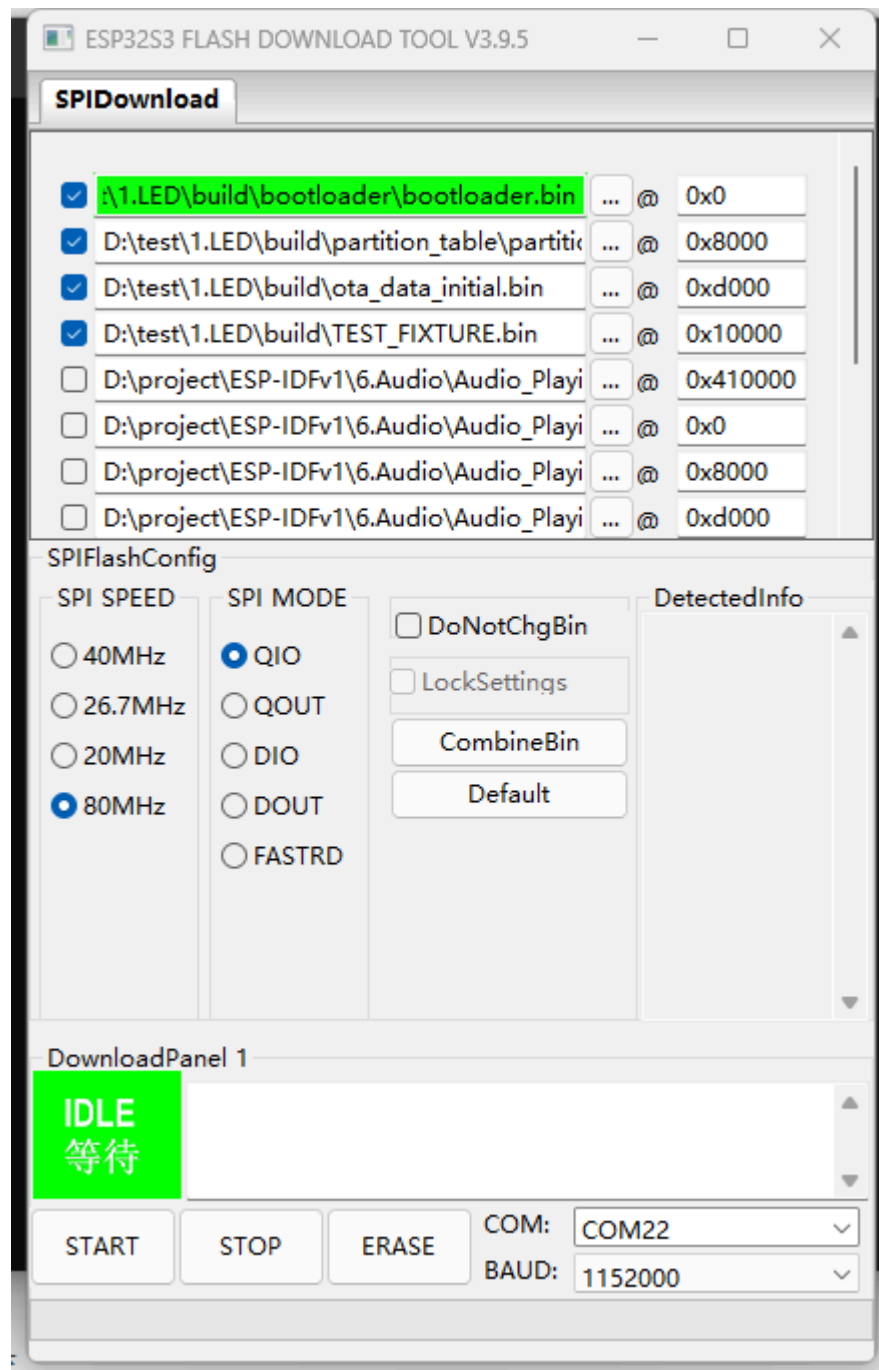
Default

DetectedInfo

Parameter	Recommended Value	Description
SPI SPEED	80MHz	SPI Flash clock frequency
SPI MODE	QIO	Flash communication mode (Quad I/O)
FLASH SIZE	16Mbit (N16R8)	Select according to actual Flash capacity
BAUD	1152000	Flash baud rate

Step 4: Connect & Flash

1. Select the development board's corresponding **COM Port**



2. Set baud rate to **1152000**
3. Click **START** to begin flashing

4. When the progress bar is full and shows **FINISH**, it's complete

Step 5: Manually Enter Download Mode

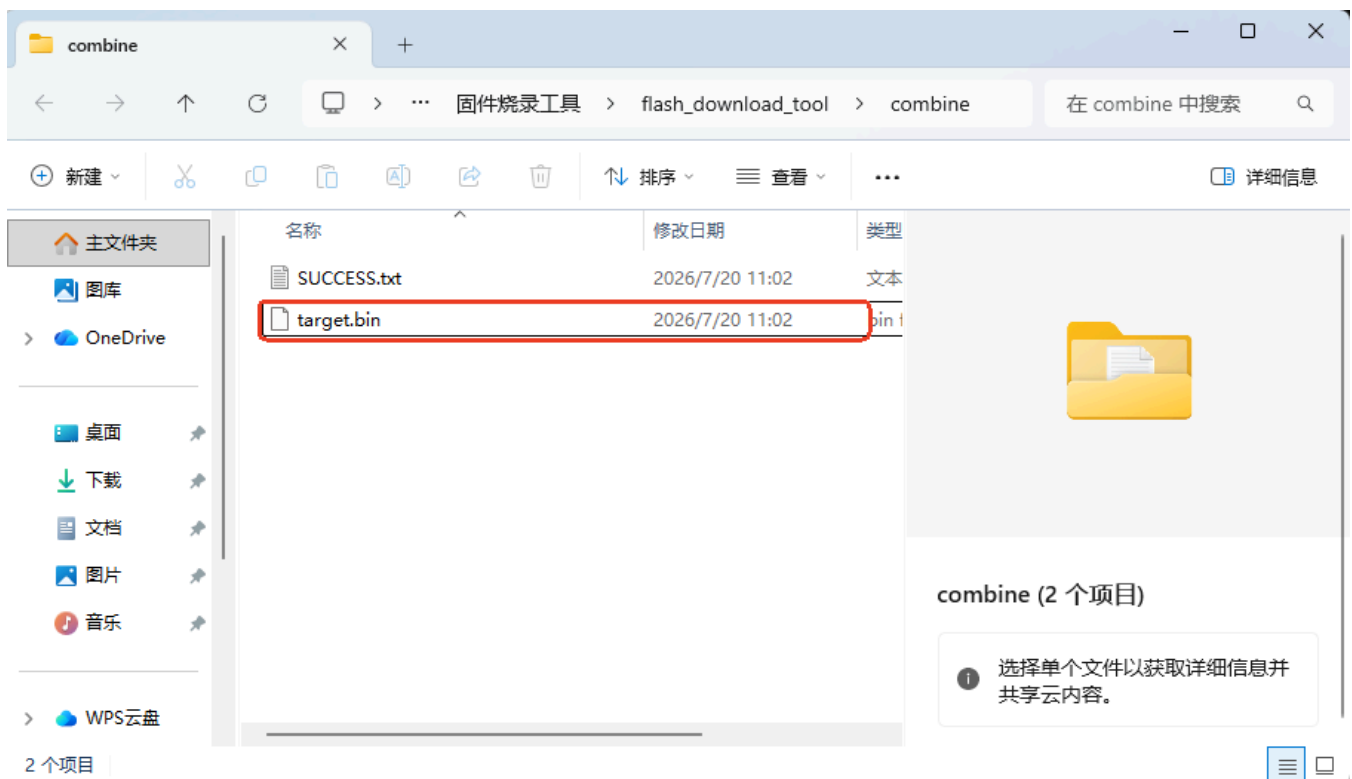
If the tool keeps displaying `waiting for chip to power up`, you need to manually put the chip into download mode:

1. **Hold the BOOT button**
2. **Press the RST button once** (reset)
3. **Release the BOOT button**

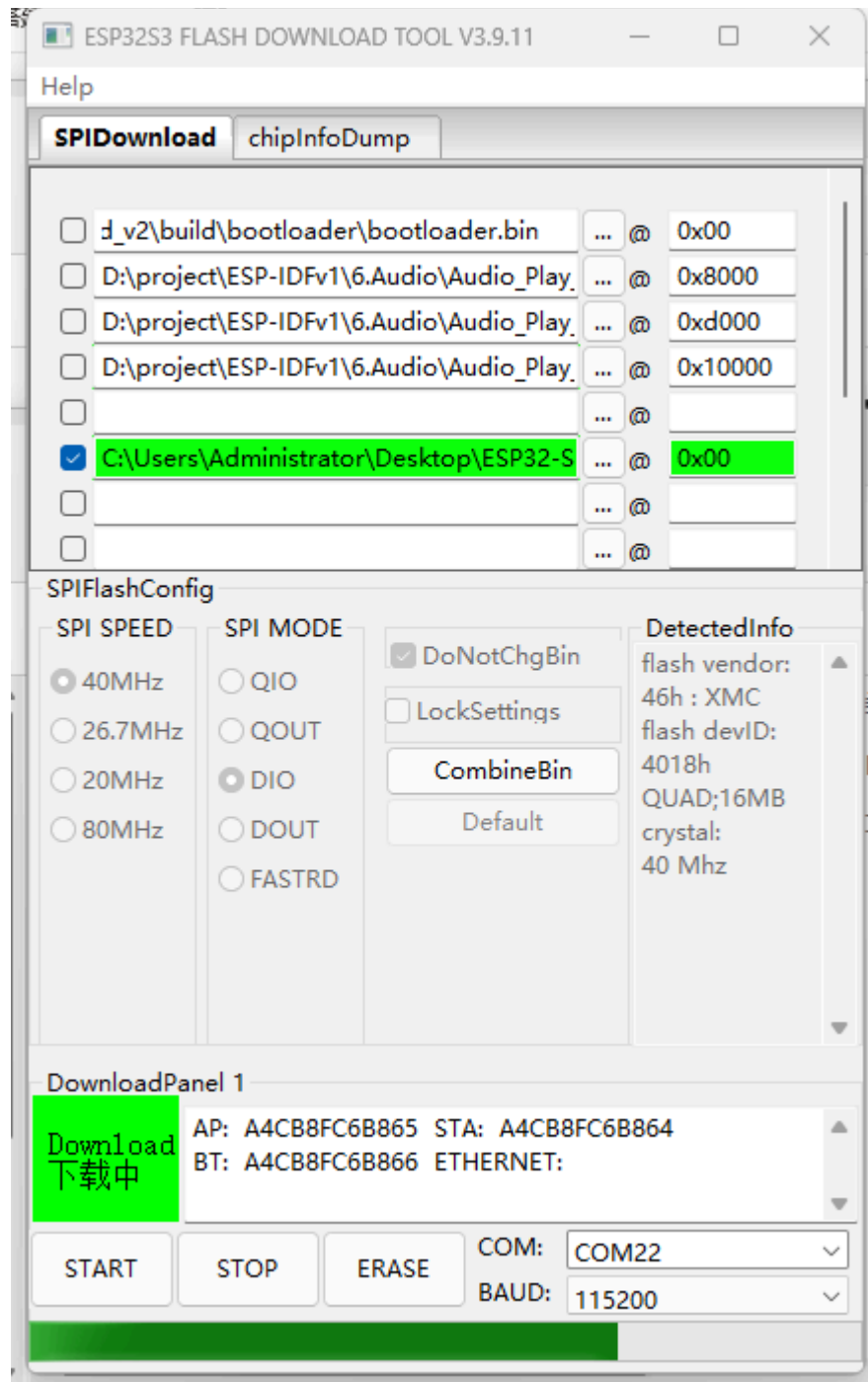
Some development boards (ESP32-S3-N16R8) require this operation, especially after running other firmware.

5.3 Merged Firmware Flashing

Click the **Merge** button on the tool interface: `CombineBin` The merge configuration dialog will appear:



You can rename the merged file (e.g., `record` and `playback.bin`):



After merging is complete, set the generated `.bin` file address to `0x0`, check it, and flash.

6. Flash File Description

A complete flash typically contains the following files:

```

build/
├─ bootloader/
│   └─ bootloader.bin      # Address 0x0      - Secondary bootloader
├─ partition_table/
│   └─ partition-table.bin # Address 0x8000 - Partition table
├─ hello_world.bin        # Address 0x10000 - Application firmware
├─ flash_args              # Flash parameters (used by esptool)
└─ hello_world.elf         # ELF file (contains debug info, not flashed)

```

7. Troubleshooting

Phenomenon	Cause	Solution
Timed out waiting for packet header	Chip not in download mode	Manual: Hold BOOT → Press RST → Release BOOT, then flash again
Invalid head of packet	Baud rate too high or driver issue	Lower baud rate: <code>idf.py -b 115200 flash</code>
A fatal error occurred: Could not open COMx	COM port occupied	Close serial assistant / other VSCode Monitor sessions
No serial port in Device Manager	USB driver missing	Install CH340 / CP2102 driver
Program doesn't run after flashing	Chip not reset	Press RST button to manually reset; or auto-reset failed after flashing
FLASH SIZE selected incorrectly	Firmware partition doesn't match hardware	For N16R8 model, select 16Mbit Flash
Flash Download Tool file path error	Path contains Chinese/spaces	Move <code>.bin</code> files to an English path (e.g., <code>D:\firmware\</code>)
Build succeeded but flashing failed	Serial port not selected or conflict	Check if VSCode bottom status bar COM port is correct
<code>idf.py: command not found</code>	Not running in EIM terminal	Use <code>F1</code> → ESP-IDF: Open ESP-IDF Terminal